

SUPPORT DE COURS COMPLET

Service web

16h CM · 8h TP · 24h

Dr. Zakaria Sawadogo

École Polytechnique de Ouagadougou



Sommaire

Syllabus

1. Chapitre 1 — Introduction aux services web et au protocole HTTP
2. Chapitre 2 — Architecture REST et conception d'API
3. Chapitre 3 — Formats d'échange : JSON, XML et SOAP
4. Chapitre 4 — Authentification et autorisation des services web
5. Chapitre 5 — Documentation et test des API
6. Chapitre 6 — Architectures orientées services et microservices

Service web

Volume horaire : 16h CM · 8h TP (24h)

Présentation de la matière

Ce module aborde la conception, le développement et l'exploitation des services web, qui constituent aujourd'hui le mode d'interopérabilité dominant entre applications, qu'il s'agisse d'applications web, mobiles ou de services internes à une organisation. Il part du protocole HTTP lui-même pour aller jusqu'aux architectures distribuées à base de microservices, en couvrant en chemin la conception d'API REST, les formats d'échange, l'authentification, et la documentation/test d'API. Il constitue un prérequis technique naturel pour aborder en profondeur la sécurisation des API, déjà introduite dans le module *Sécurité des applications*.

Objectifs pédagogiques

- Comprendre le fonctionnement du protocole HTTP et son rôle de socle des services web modernes.
- Concevoir une API REST respectant les principes du style architectural REST et les bonnes pratiques de conception reconnues par la profession.
- Comparer et choisir un format d'échange adapté (JSON, XML, SOAP) selon le contexte technique et historique d'un projet.
- Mettre en œuvre des mécanismes d'authentification et d'autorisation modernes (clés d'API, OAuth 2.0, JWT) et comprendre le rôle de CORS.
- Documenter formellement une API avec OpenAPI et mettre en place une stratégie de test adaptée (unitaire, intégration, contrat).
- Situer les architectures orientées services et microservices par rapport à l'architecture monolithique, et identifier les défis qu'elles introduisent.

Prérequis

Une aisance de base en programmation (module *Algorithmique et Programmation en C*, transposable aux exemples Python utilisés dans ce module) est attendue. Ce module prépare directement le chapitre « Sécurisation des API » du module *Sécurité des applications* ; il est recommandé, mais non strictement obligatoire, de le suivre en amont ou en parallèle de ce dernier.

Plan de séances

Cours magistraux (16h)

Séance	Chapitre	Durée
1	Introduction aux services web et au protocole HTTP	2h
2	Architecture REST et conception d'API	3h
3	Formats d'échange : JSON, XML et SOAP	3h
4	Authentification et autorisation des services web	3h
5	Documentation et test des API	2h
6	Architectures orientées services et microservices	3h

Travaux pratiques (8h)

Séance	Sujet	Durée
1	Découverte du protocole HTTP et d'une API REST	2h
2	Conception et implémentation d'une API REST	2h
3	Authentification par JWT d'une API	2h
4	Documentation et tests automatisés d'une API	2h

Évaluation

- TP notés : 40 %
- Mini-projet (extension de l'API développée en TP2-TP4, avec soutenance courte) : 20 %
- Examen final : 40 %

Environnement technique

Les travaux pratiques utilisent Python (bibliothèque `requests` pour les appels HTTP, `Flask` pour l'implémentation du serveur), `curl` pour l'exploration manuelle du protocole HTTP, ainsi que des outils de documentation et de test d'API (Swagger UI, `pytest`).

Bibliographie

- IETF, RFC 9110 — *HTTP Semantics* ; RFC 9112 — *HTTP/1.1*.
- IETF, RFC 6749 — *The OAuth 2.0 Authorization Framework* ; RFC 7519 — *JSON Web Token (JWT)*.
- IETF, RFC 9457 — *Problem Details for HTTP APIs*.
- R. Fielding, *Architectural Styles and the Design of Network-based Software Architectures* (thèse de doctorat, 2000).
- OpenAPI Initiative — spécification OpenAPI, spec.openapis.org.

- L. Richardson, M. Amundsen, *RESTful Web APIs*, O'Reilly.
- S. Newman, *Building Microservices*, O'Reilly.

Chapitre 1 — Introduction aux services web et au protocole HTTP

1. Qu'est-ce qu'un service web ?

Un **service web** est un mécanisme permettant à deux applications, potentiellement écrites dans des langages différents et exécutées sur des machines différentes, d'échanger des données et de déclencher des traitements à distance, à travers un réseau, en s'appuyant sur des protocoles et des formats standardisés plutôt que sur des mécanismes propriétaires. Ce module étudie principalement les services web construits au-dessus du protocole **HTTP** (*HyperText Transfer Protocol*), aujourd'hui de très loin le mode d'exposition dominant, mais le principe général — exposer une fonctionnalité de façon interopérable, indépendamment de la technologie du consommateur — est antérieur au Web lui-même.

Bref historique

Avant la généralisation des services web tels qu'on les connaît aujourd'hui, l'interopérabilité entre applications distantes reposait sur des technologies plus lourdes et souvent propriétaires ou fortement couplées à une plateforme particulière : appels de procédure à distance (RPC), CORBA, DCOM. Ces approches imposaient généralement que les deux parties partagent des bibliothèques, des définitions d'interface binaires ou, à tout le moins, une pile technologique compatible.

Le tournant est venu de deux évolutions convergentes au début des années 2000 : d'une part la normalisation de **SOAP** (*Simple Object Access Protocol*), qui proposait d'encapsuler des appels de méthode dans des messages XML transportés sur HTTP — approfondi au chapitre 3 — et d'autre part la formalisation, par Roy Fielding dans sa thèse de doctorat en 2000, du style architectural **REST** (*REpresentational State Transfer*), qui ne cherche pas à imiter un appel de procédure à distance mais à exploiter directement les propriétés du Web (ressources adressables par URI, verbes HTTP uniformes, absence d'état côté serveur). À partir du milieu des années 2000, avec la multiplication des API publiques (réseaux sociaux, plateformes de cartographie, services de paiement) puis l'essor des applications mobiles consommant des API distantes, REST s'est progressivement imposé comme l'approche dominante pour la majorité des nouveaux services web, sans toutefois faire disparaître SOAP de certains contextes historiques ou réglementés (chapitre 3).

2. Architecture client-serveur

Un service web s'inscrit dans une architecture **client-serveur** : le **client** (navigateur, application mobile, autre service, script) initie une requête vers le **serveur**, qui la traite et retourne une réponse. Cette séparation des responsabilités présente plusieurs avantages structurants, déjà rencontrés en filigrane dans d'autres modules de ce programme :

- **Indépendance d'évolution** : client et serveur peuvent évoluer séparément, tant que le contrat d'interface (le format des requêtes et réponses) reste stable ou correctement versionné (chapitre 2).

- **Réutilisabilité** : un même service peut être consommé par plusieurs clients hétérogènes (application web, application mobile, autre service back-end) sans duplication de logique métier.
- **Centralisation du traitement et des données sensibles** : la logique métier et l'accès aux données restent côté serveur, sous le contrôle de l'organisation qui les expose, plutôt que dupliqués et potentiellement exposés côté client.



3. Anatomie d'une requête et d'une réponse HTTP

HTTP est un protocole de la couche application, textuel dans ses versions historiques (HTTP/1.0, HTTP/1.1), fonctionnant selon un modèle **requête-réponse** : le client émet une requête, le serveur retourne exactement une réponse. Une requête HTTP comporte trois parties :

```

GET /api/utilisateurs/42 HTTP/1.1
Host: exemple.edu
Accept: application/json
Authorization: Bearer eyJhbGciOiJIUzI1NiIs...
  
```

- une **ligne de requête** : méthode, chemin de la ressource, version du protocole ;
- des **en-têtes** (*headers*) : métadonnées clé-valeur (format attendu, authentification, etc.) ;
- éventuellement un **corps** (*body*), absent ici pour une requête `GET`.

La réponse correspondante suit une structure symétrique :

```

HTTP/1.1 200 OK
Content-Type: application/json
Content-Length: 58

{"id": 42, "nom": "Awa Traoré", "role": "etudiant"}
  
```

- une **ligne de statut** : version du protocole, code de statut, texte explicatif ;
- des **en-têtes** de réponse ;
- un **corps** de réponse, ici au format JSON (chapitre 3).

4. Méthodes HTTP

Chaque requête HTTP porte une **méthode** (aussi appelée verbe), qui indique l'action attendue sur la ressource ciblée. Le tableau suivant présente les méthodes les plus utilisées par les services web, avec deux propriétés essentielles pour la conception d'API (approfondies au chapitre 2) :

- **sûre** (*safe*) : ne doit produire aucun effet de bord sur le serveur (lecture seule) ;

- **idempotente** : exécuter la même requête plusieurs fois de suite produit le même effet que l'exécuter une seule fois.

Méthode	Usage typique	Sûre	Idempotente
GET	Récupérer une ressource	Oui	Oui
POST	Créer une ressource, ou déclencher un traitement non idempotent	Non	Non
PUT	Remplacer intégralement une ressource existante (ou la créer à une adresse connue)	Non	Oui
PATCH	Modifier partiellement une ressource	Non	Non (en général)
DELETE	Supprimer une ressource	Non	Oui
HEAD	Comme GET, mais sans corps de réponse (vérifier l'existence, les en-têtes)	Oui	Oui
OPTIONS	Découvrir les méthodes et options supportées par une ressource	Oui	Oui

L'idempotence de `PUT` et `DELETE` mérite d'être précisée : rejouer plusieurs fois `DELETE /api/commandes/17` laisse le système dans le même état final (la ressource n'existe plus), même si la deuxième requête renvoie un code d'erreur (ressource déjà absente) plutôt qu'un succès. `POST`, en revanche, n'est en général pas idempotent : rejouer une requête de création de commande sans précaution crée typiquement une nouvelle commande à chaque appel.

5. Codes de statut HTTP

Le code de statut, sur trois chiffres, indique le résultat du traitement de la requête. Le premier chiffre définit une classe :

Classe	Signification	Exemples courants
1xx	Information	100 Continue
2xx	Succès	200 OK, 201 Created, 204 No Content
3xx	Redirection	301 Moved Permanently, 304 Not Modified
4xx	Erreur côté client	400 Bad Request, 401 Unauthorized, 403 Forbidden, 404 Not Found, 409 Conflict, 429 Too Many Requests
5xx	Erreur côté serveur	500 Internal Server Error, 502 Bad Gateway, 503 Service Unavailable

Une confusion fréquente concerne `401 Unauthorized` et `403 Forbidden` : le premier signifie que le client n'est pas authentifié (ou l'est de façon invalide), le second qu'il est authentifié mais n'a pas les droits nécessaires pour l'action demandée. Le choix précis du code de statut fait partie intégrante de la

conception d'une bonne API (chapitre 2) : un client (humain ou logiciel) doit pouvoir réagir correctement sans avoir à analyser le corps de la réponse.

6. En-têtes HTTP courants

En-tête	Rôle
Content-Type	Indique le format du corps envoyé (ex. <code>application/json</code> , <code>application/xml</code>)
Accept	Indique au serveur les formats de réponse acceptés par le client
Authorization	Transporte les informations d'authentification (chapitre 4)
Cache-Control	Contrôle la mise en cache de la réponse par des intermédiaires ou le client
Location	Indique, en réponse à une création (<code>201 Created</code>), l'URI de la ressource créée
ETag	Identifiant de version d'une ressource, utilisé pour la mise en cache conditionnelle

7. Statelessness : l'absence d'état côté serveur

L'une des contraintes architecturales fondamentales de HTTP, et plus largement de REST (chapitre 2), est le caractère **sans état** (*stateless*) de chaque requête : le serveur ne conserve aucune information de contexte entre deux requêtes successives d'un même client. Chaque requête doit se suffire à elle-même et contenir toutes les informations nécessaires à son traitement (notamment les informations d'authentification, chapitre 4).

Ce principe a des conséquences pratiques importantes :

- **Scalabilité horizontale simplifiée** : n'importe quel serveur d'un pool peut traiter n'importe quelle requête d'un client, sans avoir besoin d'accéder à un état de session conservé en mémoire sur un serveur précédent.
- **Résilience** : la panne d'un serveur ne fait pas perdre le contexte applicatif du client, puisqu'aucun état n'y était stocké.
- **Report de la charge d'état sur le client** : lorsqu'un contexte doit malgré tout persister (ex. un utilisateur authentifié), il est porté par le client à chaque requête (cookie, jeton — chapitre 4), et non maintenu en mémoire côté serveur entre deux requêtes.

Il convient de distinguer l'absence d'état *de la requête* de l'absence d'état *des données* : un service web sans état côté requête interagit très couramment avec une base de données persistante (état applicatif durable), ce qui n'est pas contradictoire avec le principe de statelessness au sens HTTP/REST.

8. Exemples concrets

Requête en ligne de commande avec `curl`

```
# Requête GET simple
curl https://api.exemple.edu/utilisateurs/42

# Requête POST avec un corps JSON
curl -X POST https://api.exemple.edu/utilisateurs \
  -H "Content-Type: application/json" \
  -d '{"nom": "Awa Traoré", "role": "etudiant"}'

# Affichage des en-têtes de réponse uniquement
curl -I https://api.exemple.edu/utilisateurs/42
```

Client Python avec la bibliothèque `requests`

```
import requests

reponse = requests.get("https://api.exemple.edu/utilisateurs/42")
print(reponse.status_code)    # 200
print(reponse.json())         # {'id': 42, 'nom': 'Awa Traoré', ...}
```

Serveur minimal avec Flask

```
from flask import Flask, jsonify

app = Flask(__name__)

@app.route("/api/utilisateurs/<int:id_utilisateur>", methods=["GET"])
def obtenir_utilisateur(id_utilisateur):
    # Dans une vraie application, la donnée proviendrait d'une base de données.
    return jsonify({"id": id_utilisateur, "nom": "Awa Traoré", "role": "etudiant"}), 200

if __name__ == "__main__":
    app.run(debug=True)
```

À retenir

- Un service web permet l'interopérabilité entre applications hétérogènes ; REST, apparu au début des années 2000, s'est imposé comme l'approche dominante face à SOAP dans la majorité des nouveaux usages.
- HTTP fonctionne selon un modèle requête-réponse strict, chaque échange comportant une ligne de requête/statut, des en-têtes, et un corps optionnel.
- Les méthodes HTTP portent une sémantique précise (sûreté, idempotence) que la conception d'API doit respecter scrupuleusement (approfondi au chapitre 2).
- Les codes de statut doivent être choisis avec soin ; `401` et `403` en particulier ne sont pas interchangeables.
- Le caractère sans état de HTTP est une contrainte architecturale fondamentale de REST, avec des conséquences directes sur la scalabilité et sur la gestion de l'authentification.

Chapitre 2 — Architecture REST et conception d'API

1. REST : un style architectural, pas un protocole

REST (*REpresentational State Transfer*) n'est pas un protocole ni une norme au sens strict, mais un **style architectural** défini par Roy Fielding, décrivant un ensemble de contraintes qu'une architecture doit respecter pour être qualifiée de « RESTful ». Une API qui s'appuie sur HTTP sans respecter ces contraintes n'est pas nécessairement « REST » au sens plein du terme, même si l'usage courant du secteur a largement élargi le sens du mot pour désigner, de façon plus lâche, toute API HTTP orientée ressources.

Les contraintes principales du style REST sont :

- **Client-serveur** : séparation des responsabilités (chapitre 1).
- **Sans état** (*stateless*) : chaque requête se suffit à elle-même (chapitre 1).
- **Cache** : les réponses doivent indiquer explicitement si elles sont cacheables, pour permettre au client ou à un intermédiaire d'éviter des requêtes redondantes.
- **Interface uniforme** : un ensemble réduit et cohérent de conventions (identification des ressources par URI, manipulation via des représentations, messages auto-descriptifs, HATEOAS — détaillés ci-dessous) appliqué uniformément à toutes les ressources.
- **Système en couches** : le client ne doit pas savoir s'il communique directement avec le serveur final ou avec un intermédiaire (proxy, passerelle, répartiteur de charge — cf. chapitre 6 sur l'API Gateway).
- **Code à la demande** (optionnel) : le serveur peut, de façon facultative, transmettre du code exécutable par le client (rarement utilisé pour des API de données).

2. Le modèle de maturité de Richardson

Le **Richardson Maturity Model**, popularisé par Leonard Richardson, offre une grille de lecture pédagogique pour évaluer à quel point une API HTTP se rapproche des principes REST, en quatre niveaux :

Niveau	Caractéristique	Exemple
0	Un point d'entrée unique, un seul verbe HTTP (souvent <code>POST</code>), le reste de la logique encodé dans le corps de la requête (proche d'un appel RPC déguisé en HTTP)	<code>POST /api</code> avec <code>{"action": "obtenir_utilisateur", "id": 42}</code>
1	Introduction de ressources distinctes, identifiées par des URI différentes, mais toujours avec un seul verbe HTTP	<code>POST /api/utilisateurs/42</code>
2	Utilisation correcte des verbes HTTP (<code>GET</code> , <code>POST</code> , <code>PUT</code> , <code>DELETE</code> ...) et des codes de statut pour chaque ressource	<code>GET /api/utilisateurs/42</code> → <code>200 OK</code>

Niveau	Caractéristique	Exemple
3	Ajout de HATEOAS : les réponses contiennent des liens hypermédia permettant au client de découvrir dynamiquement les actions disponibles	Réponse incluant un lien vers l'action d'annulation d'une commande

La grande majorité des API REST rencontrées en pratique se situent au niveau 2 : ressources bien identifiées, verbes et codes de statut correctement utilisés, mais sans hypermédia. Le niveau 3 reste rare en dehors de contextes spécifiques, la valeur ajoutée de HATEOAS étant souvent jugée disproportionnée par rapport à sa complexité de mise en œuvre.

3. Ressources et URI

Le concept central de REST est la **ressource** : toute entité manipulable (un utilisateur, une commande, une collection de commandes) identifiée de façon unique par un **URI** (*Uniform Resource Identifier*). La conception d'une bonne API repose largement sur le choix de nommage des URI :

- Utiliser des **noms**, pas des verbes, dans les chemins (l'action est portée par la méthode HTTP) : `GET /commandes/17`, et non `GET /obtenirCommande?id=17`.
- Utiliser le **pluriel** pour les collections : `/utilisateurs` (collection), `/utilisateurs/42` (élément).
- Représenter les relations par imbrication lorsqu'elle a un sens métier clair : `/utilisateurs/42/commandes` (commandes de l'utilisateur 42).
- Éviter les chemins profondément imbriqués (plus de deux ou trois niveaux), qui rigidifient inutilement l'API et couplent fortement le client à une structure de données particulière.

Bonne pratique	À éviter
<code>GET /commandes/17</code>	<code>GET /getCommande/17</code>
<code>POST /commandes</code>	<code>POST /commandes/creer</code>
<code>DELETE /commandes/17</code>	<code>POST /commandes/17/supprimer</code>
<code>GET /utilisateurs/42/commandes</code>	<code>GET /commandes?utilisateur=42&action=liste</code>

4. HATEOAS

HATEOAS (*Hypermedia As The Engine Of Application State*) est la contrainte selon laquelle un client ne devrait pas avoir besoin de connaître à l'avance la structure complète de l'API : chaque réponse contient les liens vers les actions ou ressources pertinentes disponibles depuis l'état courant, de façon analogue à la navigation par hyperliens d'une page web.

```
{
  "id": 17,
  "statut": "en_preparation",
  "montant": 45000,
  "_liens": {
    "self": {"href": "/commandes/17"},
    "annuler": {"href": "/commandes/17/annulation", "methode": "POST"},
    "utilisateur": {"href": "/utilisateurs/42"}
  }
}
```

En pratique, la plupart des API REST industrielles n'implémentent pas HATEOAS de façon stricte, pour des raisons de complexité de mise en œuvre et parce que les clients modernes (SDK générés, documentation OpenAPI — chapitre 5) découvrent le contrat d'API par d'autres moyens que la navigation hypermédia à l'exécution. Sa connaissance reste néanmoins utile pour comprendre pleinement le style REST originel et évaluer les compromis effectués par une API donnée.

5. Versionnement d'une API

Une API évolue dans le temps ; le versionnement permet de faire évoluer son contrat sans casser brutalement les clients existants. Plusieurs approches coexistent, chacune avec ses compromis :

Approche	Exemple	Avantage	Inconvénient
Dans l'URI	<code>/api/v2/commandes</code>	Simple, visible, facile à router	« Pollue » l'URI d'une information qui n'est pas, au sens strict, une propriété de la ressource
En-tête personnalisé	<code>X-API-Version: 2</code>	URI stable	Moins visible, nécessite une documentation explicite
Négociation de contenu	<code>Accept: application/vnd.exemple.v2+json</code>	Conforme à l'esprit HTTP (négociation de représentation)	Plus complexe à mettre en œuvre et à documenter côté client

Le versionnement par l'URI reste le plus répandu en pratique pour sa simplicité opérationnelle, malgré son entorse à la pureté du style REST (une ressource devrait, en théorie, garder la même identité indépendamment de sa représentation).

6. Pagination, filtrage et tri

Une collection potentiellement volumineuse (ex. des milliers de commandes) ne doit jamais être renvoyée intégralement en une seule réponse. La **pagination** est indispensable dès la conception :

```
GET /commandes?page=2&taille_page=20
GET /commandes?curseur=eyJpZCI6MTAwfQ&limite=20
```

- La **pagination par numéro de page** (*offset-based*) est simple à implémenter mais peut produire des incohérences si des éléments sont ajoutés ou supprimés entre deux appels.
- La **pagination par curseur** (*cursor-based*) référence un point précis dans l'ensemble ordonné des résultats et reste stable même en cas de modifications concurrentes, au prix d'une implémentation plus complexe.

Une réponse paginée gagne à inclure des métadonnées explicites :

```
{
  "donnees": [...],
  "pagination": {
    "page": 2,
    "taille_page": 20,
    "total": 187,
    "pages_totales": 10
  }
}
```

Le filtrage (GET /commandes?statut=en_preparation) et le tri (GET /commandes?tri=-date_creation) suivent la même logique : exprimés via des paramètres de requête, jamais via de nouveaux verbes ou de nouvelles routes dédiées.

7. Exemple de conception d'API

Pour une ressource « commande », une API REST bien conçue au niveau 2 du modèle de Richardson pourrait exposer :

Méthode	URI	Action
GET	/commandes	Lister les commandes (paginé)
POST	/commandes	Créer une commande
GET	/commandes/{id}	Obtenir une commande précise
PUT	/commandes/{id}	Remplacer intégralement une commande
PATCH	/commandes/{id}	Modifier partiellement une commande (ex. changer son statut)
DELETE	/commandes/{id}	Supprimer une commande
GET	/utilisateurs/{id}/commandes	Lister les commandes d'un utilisateur donné

```

from flask import Flask, jsonify, request

app = Flask(__name__)
commandes = {}

@app.route("/commandes", methods=["GET"])
def lister_commandes():
    page = int(request.args.get("page", 1))
    taille_page = int(request.args.get("taille_page", 20))
    valeurs = list(commandes.values())
    debut = (page - 1) * taille_page
    return jsonify({
        "donnees": valeurs[debut:debut + taille_page],
        "pagination": {"page": page, "taille_page": taille_page, "total": len(valeurs)}
    })

@app.route("/commandes/<int:id_commande>", methods=["PATCH"])
def modifier_commande(id_commande):
    if id_commande not in commandes:
        return jsonify({"erreur": "Commande introuvable"}), 404
    commandes[id_commande].update(request.get_json())
    return jsonify(commandes[id_commande]), 200

```

À retenir

- REST est un style architectural défini par un ensemble de contraintes (client-serveur, sans état, cache, interface uniforme, système en couches), pas un simple synonyme d'« API HTTP ».
- Le modèle de maturité de Richardson distingue quatre niveaux ; la grande majorité des API industrielles se situent au niveau 2 (ressources, verbes, codes de statut corrects).
- Les URI doivent identifier des ressources par des noms, jamais des actions par des verbes ; l'action est portée par la méthode HTTP.
- HATEOAS reste peu répandu en pratique malgré son rôle central dans la définition originelle de REST.
- Versionnement, pagination, filtrage et tri sont des préoccupations de conception à anticiper dès la définition du contrat d'API, et non à ajouter a posteriori.

Chapitre 3 — Formats d'échange : JSON, XML et SOAP

1. Le rôle d'un format d'échange

Le format d'échange définit la façon dont les données transportées par une requête ou une réponse HTTP (chapitre 1) sont structurées et sérialisées. Le choix du format conditionne la lisibilité, la taille des messages, la richesse de la validation possible, et l'écosystème d'outils disponible. Ce chapitre compare les trois formats les plus rencontrés dans les services web : **JSON**, **XML**, et le protocole historique **SOAP**, qui s'appuie sur XML.

2. JSON

JSON (*JavaScript Object Notation*) est aujourd'hui le format dominant des API REST modernes (chapitre 2), pour sa légèreté syntaxique et sa correspondance directe avec les structures de données natives de la plupart des langages de programmation (objets/dictionnaires, tableaux/listes, chaînes, nombres, booléens, valeur nulle).

```
{
  "id": 42,
  "nom": "Awa Traoré",
  "actif": true,
  "role": "etudiant",
  "cours_suivis": ["Service web", "Cryptographie"],
  "adresse": {
    "ville": "Ouagadougou",
    "pays": "Burkina Faso"
  }
}
```

Avantages structurants de JSON par rapport à XML :

- **Concision** : absence de balises fermantes redondantes, ce qui réduit la taille des messages.
- **Correspondance directe** avec les types natifs des langages de programmation les plus courants, ce qui simplifie la sérialisation/désérialisation.
- **Lisibilité** pour un humain, tout en restant facile à analyser (*parser*) pour une machine.

JSON Schema

JSON Schema est une spécification permettant de décrire formellement la structure attendue d'un document JSON (types, champs obligatoires, contraintes de valeur), utile pour valider automatiquement une requête ou documenter précisément un contrat d'API (approfondi au chapitre 5, où OpenAPI s'appuie directement sur ce mécanisme).

```
{
  "type": "object",
  "required": ["id", "nom"],
  "properties": {
    "id": {"type": "integer"},
    "nom": {"type": "string", "minLength": 1},
    "role": {"type": "string", "enum": ["etudiant", "enseignant", "administrateur"]}
  }
}
```

3. XML

XML (*eXtensible Markup Language*) est un format à balises, plus verbeux que JSON, mais offrant des capacités de validation et de description structurelle historiquement plus riches, ainsi qu'un support natif de mécanismes absents de JSON comme les espaces de noms (*namespaces*) et les commentaires.

```
<utilisateur>
  <id>42</id>
  <nom>Awa Traoré</nom>
  <actif>true</actif>
  <role>etudiant</role>
  <adresse>
    <ville>Ouagadougou</ville>
    <pays>Burkina Faso</pays>
  </adresse>
</utilisateur>
```

XSD (XML Schema Definition)

Le pendant de JSON Schema pour XML est **XSD**, un langage permettant de définir formellement la structure, les types et les contraintes d'un document XML.

```
<xs:element name="utilisateur">
  <xs:complexType>
    <xs:sequence>
      <xs:element name="id" type="xs:integer"/>
      <xs:element name="nom" type="xs:string"/>
      <xs:element name="role">
        <xs:simpleType>
          <xs:restriction base="xs:string">
            <xs:enumeration value="etudiant"/>
            <xs:enumeration value="enseignant"/>
          </xs:restriction>
        </xs:simpleType>
      </xs:element>
    </xs:sequence>
  </xs:complexType>
</xs:element>
```

JSON et XML côte à côte

Critère	JSON	XML
Concision	Élevée	Plus verbeux
Espaces de noms	Absents	Supportés nativement
Schéma de validation	JSON Schema	XSD
Transformation de document	Limitée	Riche (XSLT)
Adoption dans les API REST modernes	Très majoritaire	Marginale, souvent héritée
Adoption dans SOAP	Non applicable	Systématique

4. SOAP

SOAP (*Simple Object Access Protocol*) est un protocole d'échange de messages structuré, apparu à la fin des années 1990, reposant intégralement sur XML et pouvant être transporté sur plusieurs protocoles (le plus souvent HTTP, mais pas exclusivement). Contrairement à REST, SOAP définit un format d'enveloppe de message strict :

```
<soap:Envelope xmlns:soap="http://www.w3.org/2003/05/soap-envelope">
  <soap:Header>
    <!-- métadonnées : authentication, transaction, etc. -->
  </soap:Header>
  <soap:Body>
    <ObtenirUtilisateur xmlns="http://exemple.edu/services">
      <id>42</id>
    </ObtenirUtilisateur>
  </soap:Body>
</soap:Envelope>
```

WSDL

Un service SOAP est décrit par un document **WSDL** (*Web Services Description Language*), qui définit formellement et exhaustivement les opérations disponibles, les types de messages échangés (souvent via un schéma XSD associé) et l'adresse du service. Cette description rigide permet une génération automatique de code client dans de nombreux langages, ce qui a longtemps constitué l'un des attraits majeurs de SOAP dans les environnements d'entreprise fortement outillés (ex. Java/.NET).

Pourquoi SOAP reste employé dans certains contextes

Bien que largement supplanté par REST pour les nouveaux développements, SOAP demeure présent dans des contextes précis, généralement pour des raisons de continuité historique et d'exigences propres à certains secteurs :

- **Systèmes hérités** (*legacy*) dans les grandes organisations (banques, administrations, compagnies aériennes), où le coût de migration d'une interface stable et fonctionnelle n'est pas justifié par les seuls avantages de REST.
- **Contrats formels stricts** : la rigueur du WSDL et la vérifiabilité poussée par schéma XSD sont parfois recherchées dans des échanges interentreprises à fort enjeu contractuel.
- **Fonctionnalités standardisées avancées** (transactions distribuées, fiabilité des messages, sécurité au niveau du message plutôt qu'au niveau du transport), définies par des extensions du protocole (WS-Security, WS-ReliableMessaging), utiles dans des contextes d'intégration d'entreprise complexes que REST ne couvre pas nativement de façon aussi normalisée.

5. Sérialisation et désérialisation

La **sérialisation** transforme une structure de données en mémoire (objet, dictionnaire) en un flux de texte ou d'octets transportable ; la **désérialisation** effectue l'opération inverse à la réception.

```
import json

# Sérialisation : structure Python → chaîne JSON
utilisateur = {"id": 42, "nom": "Awa Traoré", "role": "etudiant"}
texte_json = json.dumps(utilisateur)

# Désérialisation : chaîne JSON → structure Python
donnees = json.loads(texte_json)
print(donnees["nom"]) # Awa Traoré
```

Un point de vigilance : la désérialisation de données non fiables

Comme évoqué dans le module *Sécurité des applications* (catégorie « défaillances d'intégrité des données et logiciels » de l'OWASP Top 10), désérialiser une donnée provenant d'une source non fiable peut, selon le langage et la bibliothèque utilisés, exposer à une exécution de code arbitraire si le mécanisme de désérialisation est capable de reconstruire des objets complexes plutôt que de simples structures de données. JSON, dans son usage standard (`json.loads` en Python, `JSON.parse` en JavaScript), ne présente pas ce risque car il ne reconstruit que des types de données primitifs ; le risque apparaît surtout avec des mécanismes de sérialisation plus « riches » propres à certains langages (ex. `pickle` en Python), qu'il convient de ne jamais utiliser sur une donnée reçue d'un client non authentifié ou non fiable.

À retenir

- JSON s'est imposé comme le format dominant des API REST modernes pour sa concision et sa correspondance directe avec les structures de données natives des langages courants.
- XML reste plus verbeux mais offre des capacités historiquement plus riches (espaces de noms, transformations, schémas XSD) ; JSON Schema en constitue l'équivalent côté JSON.
- SOAP est un protocole XML strict, décrit par un contrat WSDL, encore présent dans des systèmes hérités ou des contextes d'intégration d'entreprise à exigences formelles fortes.

- S rialisation et d s rialisation sont les op rations sym triques de conversion entre structures en m moire et repr sentation transportable ; la d s rialisation de donn es non fiables via des m canismes « riches » est une source de vuln rabilit  connue.

Chapitre 4 — Authentification et autorisation des services web

1. Authentification et autorisation : deux notions distinctes

- **Authentification** : vérifier *qui* émet une requête (un utilisateur, une application tierce).
- **Autorisation** : vérifier *ce que* cet émetteur, une fois identifié, a le droit de faire.

Cette distinction, déjà rencontrée dans le module *Sécurité des applications* (catégorie « défaillances d'identification et d'authentification » de l'OWASP Top 10), est particulièrement structurante pour les services web : une API peut authentifier correctement un client tout en échouant à vérifier son autorisation sur une ressource précise (vulnérabilité de type contrôle d'accès défaillant au niveau des objets, ou BOLA), ce qui reste l'une des causes les plus fréquentes de compromission d'API en pratique.

2. Clés d'API

Une **clé d'API** (*API key*) est un identifiant statique, généralement long et aléatoire, transmis par le client à chaque requête pour s'identifier auprès du service.

```
curl https://api.exemple.edu/donnees \  
-H "X-API-Key: 7f3a9c1e-4b2d-4e8a-9c31-5d6f7a8b9c0d"
```

Les clés d'API sont simples à mettre en œuvre mais présentent des limites importantes : elles n'identifient généralement qu'une application ou un compte, sans granularité fine par utilisateur final ; elles n'expirent pas automatiquement sauf mécanisme dédié ; leur compromission (fuite dans un dépôt de code, journal, ou message d'erreur) donne un accès prolongé tant qu'elle n'est pas révoquée manuellement. Elles restent adaptées à des cas d'usage simples (identification d'une application tierce, quotas d'usage) plutôt qu'à l'authentification d'utilisateurs finaux.

3. Authentification HTTP Basic

L'authentification **Basic**, définie dans le protocole HTTP lui-même, transmet un identifiant et un mot de passe encodés en Base64 dans l'en-tête `Authorization` :

```
Authorization: Basic YXdhLnRyYW9yZTpzZWNYZXQxMjM=
```

Un point de vigilance essentiel, souvent mal compris par les étudiants découvrant ce mécanisme : **l'encodage Base64 n'est pas un chiffrement**. Il s'agit d'une simple transformation réversible sans aucune clé secrète ; quiconque intercepte cet en-tête peut retrouver l'identifiant et le mot de passe en clair immédiatement. L'authentification Basic n'est donc **sûre que sur une connexion chiffrée par**

TLS (HTTPS), condition qui doit toujours être vérifiée avant toute utilisation de ce mécanisme en production.

4. OAuth 2.0

OAuth 2.0 est un cadre d'autorisation (et non, à proprement parler, un protocole d'authentification) normalisé par l'IETF, qui permet à une application tierce d'accéder à des ressources appartenant à un utilisateur, hébergées par un autre service, sans que l'utilisateur ait à partager son mot de passe avec l'application tierce.

Les acteurs d'OAuth 2.0

Acteur	Rôle
Propriétaire de la ressource (<i>resource owner</i>)	L'utilisateur final, propriétaire des données
Client	L'application tierce souhaitant accéder aux données
Serveur d'autorisation	Authentifie l'utilisateur et délivre les jetons
Serveur de ressources	Héberge les données protégées et vérifie les jetons présentés

Principaux types d'octroi (*grant types*)

Grant type	Cas d'usage typique
Authorization Code (avec PKCE)	Applications web et mobiles avec interaction utilisateur — le flux recommandé aujourd'hui pour la quasi-totalité des cas
Client Credentials	Communication de service à service, sans utilisateur final impliqué
Refresh Token	Obtenir un nouveau jeton d'accès sans redemander l'authentification complète de l'utilisateur

Le flux *Authorization Code* se déroule schématiquement ainsi : l'application redirige l'utilisateur vers le serveur d'autorisation, celui-ci authentifie l'utilisateur et lui demande son consentement, puis redirige vers l'application avec un code temporaire à usage unique, que l'application échange ensuite, côté serveur, contre un jeton d'accès. L'extension **PKCE** (*Proof Key for Code Exchange*) ajoute une vérification cryptographique supplémentaire empêchant l'interception de ce code par une application malveillante sur le même appareil, et est aujourd'hui recommandée même pour les applications côté serveur.

Les *grant types* historiques *Implicit* et *Resource Owner Password Credentials* sont aujourd'hui déconseillés par les recommandations de sécurité les plus récentes autour d'OAuth 2.0, au profit d'*Authorization Code* avec PKCE dans la quasi-totalité des cas.

5. JWT (JSON Web Token)

Un **JWT** (*JSON Web Token*) est un format standardisé de jeton compact, auto-porteur (*self-contained*), fréquemment utilisé comme jeton d'accès dans les architectures OAuth 2.0 ou comme mécanisme d'authentification autonome. Un JWT se compose de trois parties encodées en Base64URL, séparées par des points :

```
en-tête.charge_utile.signature
```

```
// En-tête (algorithme et type)
{"alg": "HS256", "typ": "JWT"}

// Charge utile (claims)
{"sub": "42", "nom": "Awa Traoré", "role": "etudiant", "exp": 1893456000}
```

Structure et vérification

```
import jwt # bibliothèque PyJWT
import datetime

cle_secrete = "cle-tres-longue-et-aleatoire-stockee-hors-du-code"

# Génération d'un jeton
jeton = jwt.encode(
    {"sub": "42", "role": "etudiant",
     "exp": datetime.datetime.utcnow() + datetime.timedelta(hours=1)},
    cle_secrete, algorithm="HS256"
)

# Vérification d'un jeton reçu
try:
    charge_utile = jwt.decode(jeton, cle_secrete, algorithms=["HS256"])
except jwt.ExpiredSignatureError:
    charge_utile = None # jeton expiré : accès refusé
```

La **signature** garantit l'intégrité du jeton : toute modification de l'en-tête ou de la charge utile invalide la signature, détectable lors de la vérification. Il est essentiel de noter que la charge utile d'un JWT signé (mais non chiffré) est **lisible par quiconque intercepte le jeton** — elle n'est pas confidentielle, seulement protégée en intégrité. Aucune donnée sensible (mot de passe, information personnelle non destinée à être exposée) ne doit donc y figurer.

Bonnes pratiques JWT

- Toujours définir une expiration (**exp**) courte, et prévoir un mécanisme de jeton de rafraîchissement (*refresh token*) pour renouveler l'accès sans réauthentification complète.
- Ne jamais faire confiance à l'algorithme annoncé dans l'en-tête du jeton reçu (imposer explicitement, côté serveur, l'algorithme attendu lors de la vérification) — une vulnérabilité

historiquement rencontrée dans plusieurs bibliothèques JWT consistait précisément à faire confiance à un en-tête `alg` modifié par un attaquant (ex. bascule vers `none`).

- Prévoir un mécanisme de révocation (liste de révocation, ou jetons de courte durée de vie combinés à un rafraîchissement contrôlé), car un JWT signé reste valide jusqu'à son expiration même si le compte associé est désactivé entre-temps.
- Stocker la clé de signature (ou la clé privée en cas de signature asymétrique) hors du code source, dans un gestionnaire de secrets dédié.

6. CORS

CORS (*Cross-Origin Resource Sharing*) est un mécanisme du navigateur, complémentaire de l'authentification, qui contrôle si une page web chargée depuis une origine donnée est autorisée à effectuer des requêtes vers une API hébergée sur une origine différente. Sans CORS, la politique par défaut du navigateur (*same-origin policy*) bloquerait toute requête cross-origin.

```
Access-Control-Allow-Origin: https://app.exemple.edu
Access-Control-Allow-Methods: GET, POST, PUT, DELETE
Access-Control-Allow-Headers: Authorization, Content-Type
Access-Control-Allow-Credentials: true
```

Un point de vigilance essentiel, déjà souligné dans le module *Sécurité des applications* : configurer `Access-Control-Allow-Origin: *` (toute origine autorisée) tout en acceptant des identifiants (cookies, en-têtes d'authentification) constitue une mauvaise pratique de sécurité, potentiellement exploitable par un site malveillant pour effectuer des requêtes authentifiées au nom d'un utilisateur. La configuration correcte consiste à lister explicitement les origines légitimes autorisées.

```
from flask import Flask
from flask_cors import CORS

app = Flask(__name__)
CORS(app, origins=["https://app.exemple.edu"], supports_credentials=True)
```

À retenir

- Authentification (qui est l'émetteur) et autorisation (ce qu'il a le droit de faire) sont deux vérifications distinctes, qui doivent être appliquées systématiquement, y compris pour chaque objet individuel accédé.
- Les clés d'API conviennent à l'identification d'applications ; l'authentification Basic n'est sûre que sur une connexion TLS ; OAuth 2.0 permet une délégation d'accès sans partage de mot de passe.
- Le flux *Authorization Code* avec PKCE est aujourd'hui le grant type OAuth 2.0 recommandé pour la quasi-totalité des cas d'usage impliquant un utilisateur final.
- Un JWT est signé (intégrité garantie) mais non chiffré par défaut (contenu lisible) ; l'algorithme de vérification doit toujours être imposé côté serveur, jamais déduit de l'en-tête du jeton reçu.

- CORS protège le navigateur, pas le serveur ; une configuration `Access-Control-Allow-Origin: *` combinée à des identifiants transmis constitue une mauvaise pratique de sécurité.

Chapitre 5 — Documentation et test des API

1. Pourquoi documenter une API formellement

Une API mal documentée, ou documentée uniquement de façon informelle (page wiki non maintenue, exemples éparpillés), impose à chaque nouveau consommateur — humain ou équipe — de redécouvrir son comportement par expérimentation, source d'erreurs et de perte de temps. Une **documentation formelle et exécutable**, générée à partir d'une spécification structurée plutôt que rédigée manuellement en texte libre, présente des avantages décisifs : elle reste synchronisée avec le code si elle est intégrée au processus de développement, elle permet de générer automatiquement des clients dans plusieurs langages, et elle sert de base à des tests de contrat (section 3).

2. La spécification OpenAPI

OpenAPI (anciennement connue sous le nom de Swagger, avant que sa spécification ne soit cédée à l'*OpenAPI Initiative*) est aujourd'hui la spécification de référence pour décrire formellement une API REST au format JSON ou YAML : ses ressources, ses opérations, ses paramètres, les schémas de ses corps de requête et de réponse (en s'appuyant sur JSON Schema, chapitre 3), ses codes de statut possibles, et ses mécanismes d'authentification.

```

openapi: 3.0.3
info:
  title: API de gestion des commandes
  version: "1.0.0"
paths:
  /commandes/{id}:
    get:
      summary: Obtenir une commande par son identifiant
      parameters:
        - name: id
          in: path
          required: true
          schema:
            type: integer
      responses:
        "200":
          description: Commande trouvée
          content:
            application/json:
              schema:
                $ref: "#/components/schemas/Commande"
        "404":
          description: Commande introuvable
components:
  schemas:
    Commande:
      type: object
      required: [id, statut, montant]
      properties:
        id:
          type: integer
        statut:
          type: string
          enum: [en_preparation, expediee, livree, annulee]
        montant:
          type: number

```

Ce que permet une spécification OpenAPI

- **Documentation interactive** : des outils (ex. Swagger UI, Redoc) génèrent automatiquement une interface web navigable et testable à partir du fichier de spécification.
- **Génération de code** : génération automatique de clients (SDK) dans de nombreux langages, et parfois de squelettes de serveur, à partir de la même source de vérité.
- **Validation automatique** : une requête ou une réponse peut être validée automatiquement contre le schéma déclaré, détectant les écarts entre l'implémentation réelle et le contrat documenté.
- **Base pour les tests de contrat** (section 3).

Documentation générée depuis le code

De nombreux frameworks modernes (ex. FastAPI en Python) génèrent automatiquement une spécification OpenAPI à partir du code de l'application lui-même (annotations de type, modèles de

données), réduisant le risque de désynchronisation entre le code et sa documentation, comparé à une spécification maintenue manuellement en parallèle du code.

```
from fastapi import FastAPI
from pydantic import BaseModel

app = FastAPI(title="API de gestion des commandes")

class Commande(BaseModel):
    id: int
    statut: str
    montant: float

@app.get("/commandes/{id_commande}", response_model=Commande)
def obtenir_commande(id_commande: int):
    return {"id": id_commande, "statut": "en_preparation", "montant": 45000}

# La spécification OpenAPI est disponible automatiquement sur /openapi.json,
# et une interface interactive Swagger UI sur /docs.
```

3. Stratégies de test d'une API

Comme pour tout logiciel, les tests d'une API se déclinent en plusieurs niveaux complémentaires, chacun ayant un objectif et un coût différents.

Niveau	Objectif	Exemple
Test unitaire	Vérifier isolément une fonction ou un composant métier, sans passer par la couche HTTP	Vérifier qu'une fonction de calcul de remise applique correctement le taux attendu
Test d'intégration	Vérifier le comportement de l'API à travers de véritables requêtes HTTP, généralement avec une base de données de test	Envoyer une requête POST /commandes et vérifier la réponse et l'état de la base
Test de contrat (<i>contract testing</i>)	Vérifier que l'implémentation respecte fidèlement le contrat documenté (OpenAPI)	Valider automatiquement chaque réponse réelle contre le schéma déclaré
Test de bout en bout (<i>end-to-end</i>)	Vérifier un scénario complet impliquant plusieurs services ou composants	Créer un compte, s'authentifier, passer une commande, vérifier la facturation

Test d'intégration avec `pytest`

```
import pytest
from mon_application import app

@pytest.fixture
def client():
    app.config["TESTING"] = True
    with app.test_client() as client:
        yield client

def test_creation_commande(client):
    reponse = client.post("/commandes", json={"produit": "Livre", "quantite": 2})
    assert reponse.status_code == 201
    corps = reponse.get_json()
    assert corps["statut"] == "en_preparation"

def test_commande_introuvable(client):
    reponse = client.get("/commandes/99999")
    assert reponse.status_code == 404
```

Test manuel exploratoire avec des outils dédiés

Des outils comme **Postman** ou son équivalent en ligne de commande **Newman** permettent de construire des collections de requêtes réutilisables, de les enchaîner en scénarios, et de les automatiser dans un pipeline d'intégration continue, en complément — et non en remplacement — des tests automatisés écrits en code (`pytest` ou équivalent), qui restent nécessaires pour une couverture systématique et reproductible.

Test de contrat automatisé

Un test de contrat vérifie que chaque réponse réelle de l'API respecte le schéma JSON Schema déclaré dans la spécification OpenAPI, ce qui permet de détecter automatiquement une dérive entre le comportement réel de l'implémentation et sa documentation officielle — un problème fréquent lorsque documentation et code évoluent séparément.

```
from openapi_core import Spec
from openapi_core.validation.response import openapi_response_validator

spec = Spec.from_file_path("openapi.yaml")
# La validation compare la réponse HTTP réelle au schéma déclaré pour cette route
# et lève une erreur en cas d'écart (champ manquant, type incorrect, code de statut
# non documenté).
```

4. Documentation des erreurs

Une API bien documentée décrit également ses réponses d'erreur, idéalement selon un format cohérent à travers toute l'API (ex. la spécification **RFC 9457 — Problem Details for HTTP APIs**, qui normalise une structure commune pour les réponses d'erreur) :

```
{
  "type": "https://exemple.edu/erreurs/commande-introuvable",
  "titre": "Commande introuvable",
  "statut": 404,
  "detail": "Aucune commande ne correspond à l'identifiant 99999."
}
```

Documenter systématiquement les codes d'erreur possibles pour chaque opération, plutôt que de se limiter au seul cas de succès, réduit fortement le temps d'intégration côté client et le nombre d'incidents liés à une mauvaise gestion des cas d'échec.

À retenir

- OpenAPI est la spécification de référence pour décrire formellement une API REST ; elle permet documentation interactive, génération de code client et validation automatisée.
- Générer la spécification depuis le code (ex. FastAPI) réduit le risque de désynchronisation entre documentation et implémentation réelle, comparé à une spécification maintenue manuellement.
- Les tests d'une API se déclinent en plusieurs niveaux complémentaires : unitaire, intégration, contrat, bout en bout, chacun ayant un rôle distinct.
- Le test de contrat vérifie spécifiquement la conformité de l'implémentation réelle à la spécification documentée, ce qui distingue cette approche des tests d'intégration classiques.
- Documenter systématiquement les cas d'erreur, idéalement selon un format uniforme, fait partie intégrante d'une documentation d'API de qualité.

Chapitre 6 — Architectures orientées services et microservices

1. De l'architecture monolithique aux architectures distribuées

Une application **monolithique** regroupe l'ensemble de ses fonctionnalités (gestion des utilisateurs, catalogue, commandes, facturation) au sein d'une seule base de code, déployée comme une seule unité. Cette approche reste pertinente pour de nombreux projets, en particulier au démarrage : simplicité de développement, de déploiement et de débogage, absence de complexité réseau interne. Elle atteint cependant des limites lorsque l'application grandit : couplage fort entre modules qui ralentit les évolutions, impossibilité de faire évoluer indépendamment des composants ayant des besoins très différents (ex. scalabilité), et risque qu'une défaillance localisée affecte l'application entière.

Deux grandes familles d'architectures distribuées répondent, avec des philosophies différentes, à ces limites : la **SOA** (*Service-Oriented Architecture*) et les **microservices**.

2. SOA et microservices : convergences et différences

Aspect	SOA	Microservices
Granularité des services	Souvent plus large (services métier étendus)	Fine (un service par capacité métier restreinte)
Mécanisme de communication central	Souvent un bus de services d'entreprise (ESB) centralisé	Communication directe entre services, sans intermédiaire centralisé obligatoire
Gouvernance	Fréquemment centralisée (contrats, standards imposés à l'échelle de l'organisation)	Décentralisée (chaque équipe responsable de son service, y compris son choix technologique)
Partage de base de données	Parfois partagée entre services	Une base de données par service, en principe (voir section 5)
Contexte d'apparition	Grandes organisations, années 2000	Popularisées à partir du milieu des années 2010, sous l'impulsion d'entreprises du numérique à forte échelle

Les microservices peuvent être vus comme une évolution des principes SOA, poussant plus loin la décentralisation et l'autonomie des équipes, avec une préférence marquée pour des mécanismes de communication simples (HTTP/REST — chapitres 1 et 2, ou gRPC — section 3) plutôt que pour un bus centralisé lourd à opérer et à faire évoluer.

3. Communication synchrone et asynchrone

Communication synchrone

Dans un échange **synchrone**, l'appelant attend la réponse de l'appelé avant de poursuivre son traitement. REST sur HTTP (chapitres 1 et 2) en est l'exemple le plus courant entre microservices, aux côtés de **gRPC**, un cadre de communication développé par Google, reposant sur HTTP/2 et sur un format binaire compact (*Protocol Buffers*), généralement plus performant que JSON sur REST mais moins directement lisible par un humain et nécessitant une définition de contrat préalable stricte (fichier `.proto`), plus proche en cela de l'esprit de WSDL (chapitre 3) que de la flexibilité informelle de REST.

```
service ServiceCommandes {
  rpc ObtenirCommande (RequeteCommande) returns (Commande);
}

message RequeteCommande {
  int32 id = 1;
}

message Commande {
  int32 id = 1;
  string statut = 2;
  double montant = 3;
}
```

Communication asynchrone

Dans un échange **asynchrone**, l'appelant émet un message sans attendre de traitement immédiat par le destinataire, généralement via une **file de messages** (*message queue*) ou un système de diffusion d'événements (ex. Kafka, RabbitMQ pour ne citer que des technologies largement répandues dans l'industrie). Ce mode découple fortement l'émetteur du récepteur dans le temps : le récepteur peut être temporairement indisponible sans que l'émetteur échoue, et plusieurs récepteurs peuvent consommer le même événement.

```
# Producteur : publie un événement sans attendre de traitement synchrone
file_evenements.publier("commande.creee", {"id": 17, "montant": 45000})

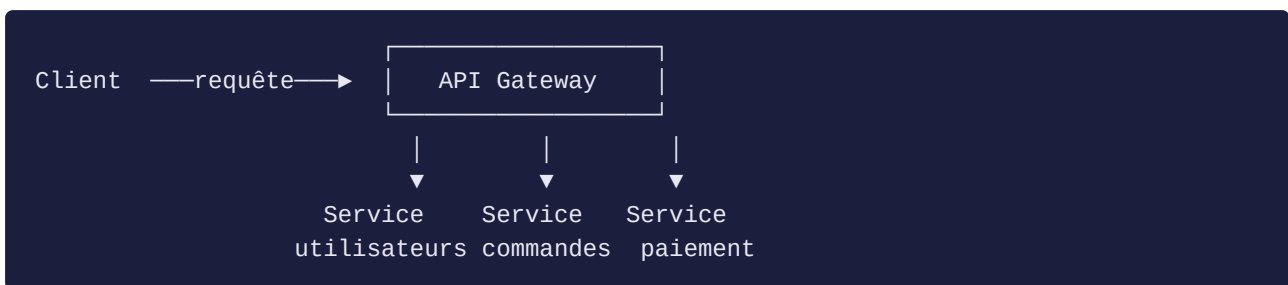
# Consommateur : traite l'événement de façon indépendante et asynchrone
def sur_commande_creee(eventement):
    envoyer_email_confirmation(eventement["id"])
```

Aspect	Synchrone (REST, gRPC)	Asynchrone (file de messages)
Couplage temporel	Fort (l'appelé doit être disponible immédiatement)	Faible (traitement différé possible)
Simplicité de raisonnement	Plus simple à suivre (un appel, une réponse)	Plus complexe (traçabilité, ordre de traitement)
Cas d'usage typique	Requête nécessitant une réponse immédiate pour l'utilisateur	Traitement différé, notification, propagation d'un événement à plusieurs consommateurs

4. API Gateway

Une **API Gateway** (passerelle d'API) est un point d'entrée unique placé devant un ensemble de microservices, qui centralise des préoccupations transverses pour éviter de les dupliquer dans chaque service :

- **Routing** des requêtes entrantes vers le microservice approprié.
- **Authentification et autorisation** centralisées (chapitre 4), évitant de réimplémenter la vérification de jetons dans chaque service.
- **Limitation de débit** (*rate limiting*) et protection contre les abus.
- **Agrégation de réponses** lorsqu'une requête cliente nécessite des données provenant de plusieurs microservices.
- **Observabilité** centralisée (journalisation, métriques) des requêtes entrantes.



Ce modèle correspond directement à la contrainte « système en couches » du style REST (chapitre 2) : le client ignore, et n'a pas besoin de savoir, qu'il communique en réalité avec une passerelle plutôt qu'avec le microservice final.

5. Découverte de services

Dans une architecture à microservices déployée à grande échelle, l'adresse réseau exacte d'une instance de service peut changer dynamiquement (redémarrage, montée en charge automatique, migration). La **découverte de services** (*service discovery*) résout ce problème : chaque service s'enregistre auprès d'un registre central (ex. Consul, ou les mécanismes intégrés d'un orchestrateur de conteneurs), et les autres services interrogent ce registre pour localiser dynamiquement une instance disponible, plutôt que de coder en dur une adresse réseau fixe.

6. Défis propres aux architectures à microservices

Le passage à une architecture à microservices n'est pas gratuit : il introduit des défis spécifiques que l'architecture monolithique ne connaît pas dans les mêmes proportions.

- **Résilience** : un appel réseau vers un autre service peut échouer (indisponibilité temporaire, latence excessive) alors qu'un appel de fonction interne à un monolithe échoue rarement. Des mécanismes dédiés (délais d'expiration stricts, réessais avec attente progressive, disjoncteurs — *circuit breakers* — qui interrompent temporairement les appels vers un service en échec répété) deviennent nécessaires.

- **Observabilité** : diagnostiquer un problème qui traverse plusieurs services nécessite des outils de traçabilité distribuée (identifiant de corrélation propagé de service en service), de journalisation centralisée, et de métriques agrégées — bien plus complexe qu'un simple examen des journaux d'un unique processus monolithique.
- **Découpage des responsabilités** (*bounded contexts*) : déterminer où placer la frontière entre deux microservices est un exercice de conception difficile ; un découpage trop fin multiplie les appels réseau et la complexité opérationnelle, un découpage trop grossier reproduit les inconvénients du monolithe sous une forme distribuée.
- **Cohérence des données** : chaque microservice possédant en principe sa propre base de données, une opération métier impliquant plusieurs services (ex. une commande qui doit débiter un stock et déclencher une facturation) ne peut plus s'appuyer sur une transaction de base de données unique classique, et nécessite des schémas de cohérence adaptés (ex. cohérence à terme, séquences d'événements compensatoires) plus complexes à raisonner qu'une transaction locale.

Quand préférer un monolithe

Il n'existe pas de supériorité universelle des microservices : pour une petite équipe, un produit encore en phase de découverte de marché, ou un domaine métier dont les frontières ne sont pas encore stabilisées, un monolithe bien structuré en modules internes cohérents reste souvent un choix plus pragmatique, la complexité opérationnelle des microservices n'étant justifiée que lorsque les gains attendus (scalabilité différenciée, autonomie d'équipes multiples, déploiement indépendant de composants) dépassent clairement son coût.

À retenir

- SOA et microservices répondent tous deux aux limites du monolithe, avec une granularité et une gouvernance différentes ; les microservices privilégient une décentralisation plus poussée.
- La communication synchrone (REST, gRPC) attend une réponse immédiate ; la communication asynchrone (files de messages) découple émetteur et récepteur dans le temps, au prix d'une complexité de raisonnement accrue.
- Une API Gateway centralise le routage, l'authentification, la limitation de débit et l'observabilité devant un ensemble de microservices, conformément à la contrainte de système en couches du style REST.
- La découverte de services permet de localiser dynamiquement des instances de service dont l'adresse réseau peut changer.
- Résilience, observabilité, découpage des responsabilités et cohérence des données sont des défis spécifiques et non négligeables des architectures à microservices, qui doivent être mis en balance avec leurs bénéfices avant d'abandonner un monolithe fonctionnel.